

연구 코드 최적화 보고서: RTMPose 기반 손 관절각 평가 파이프라인

7.1 연구 코드 소개

본 과제의 대상 코드는 손 동작 기반 로봇 텔레오퍼레이션 연구의 평가 단계에서 사용하는 `rtmpose_manus_mapping.py` 이다. 본 연구는 Manus 데이터 클러브와 RealSense 카메라로 손 동작 데이터를 수집하고, 영상 기반 손 포즈 추정 모델 (RTMPose) 을 fine-tuning 하여 Inspire Hand 의 수/의지 로봇의 실시간 텔레오퍼레이션에 활용하는 것을 목표로 한다.

대상 코드의 입력은 (1) 사람 손이 촬영된 RGB 프레임 시퀀스 (PNG) 와 (2) 같은 동작을 Manus 클러브로 동시에 기록한 관절 굴곡각 CSV(ground truth) 이며, 출력은 RTMPose 가 예측한 12 개 관절 (검지/중지/약지/소지 $\times$ MCP/PIP/DIP) 의 굴곡각과, 이를 GT 와 비교한 관절별/전체 평균 절대오차 (MAE) 이다.

전체 연구 파이프라인에서 이 코드는 “fine-tuning 한 RTMPose 가 실제 텔레오퍼레이션에 쓸 만큼 정확한가”를 정량적으로 검증하는 평가 단계를 담당한다. 즉 모델 학습 자체가 아니라, 학습된 모델의 신뢰도를 측정하는 코드이며, 이 평가가 반복적으로 (체크포인트가 바뀔 때마다, 데이터셋이 추가될 때마다) 재실행되어야 한다는 특징이 있다.

이 코드를 개선 대상으로 선택한 이유는 다음과 같다. 실제 코드베이스를 점검한 결과 `compare_new.py`, `compare_new2.py`, `compare_26.py`, `full_compare.py` 등 거의 동일한 역할을 하는 파일이 다수 존재했고, 모델이나 데이터셋을 바꿀 때마다 스크립트 전체를 복사해서 일부만 수정하는 방식으로 작업해 온 것으로 확인되었다. 이는 본 과제가 요구하는 “재사용성·확장성 문제”가 실제 연구 코드에서 그대로 드러나는 사례였기 때문에, 가장 개선 효과를 정량적으로 보여줄 수 있는 코드로 판단하였다.

7.2 기존 코드의 문제점 분석

원본 코드 (`before/rtmpose_manus_mapping.py`) 를 분석한 결과 다음과 같은 문제가 확인되었다.

책임 분리 문제. 모델 로딩, 경로/설정값 정의, CSV 로딩, 관절각 계산 함수, 추론 반복문, MAE 계산, 결과 출력이 하나의 평평한 (flat) 스크립트에 순서대로 나열되어 있다. 함수는 `calc_angle` 과 `kps_to_angles` 두 개뿐이며, 나머지 로직은 모듈 최상위 레벨에서 직접 실행된다. 이로 인해 코드의 일부만 재사용 (예: 각도 계산 로직만 다른 스크립트에서 사용) 하는 것이 불가능하고, 단위 테스트도 작성할 수 없다.

반복 계산 문제. `kps_to_angles` 함수는 12 개 관절 각각에 대해 `calc_angle` 을 개별 호출한다. 각 호출마다 벡터 차분, `np.dot`, `np.linalg.norm`, `np.arccos` 를 독립적으로 수행하므로, 동일한 연산 패턴이 12 번의 분리된 Python 함수 호출로 반복된다. 또한 손가락별로 관절이 어떻게 그룹화되는지는 코드에 명시적으로 드러나지 않고 호출 순서로만 암묵적으로 표현되어 있어, 가독성과 유지보수성이 떨어진다.

계측 부재. 추론 시간, 캐시 적중률 등 어떤 성능 지표도 측정하지 않는다. 따라서 모델이 실시간 텔레오퍼레이션에 쓸 수 있는 속도인지 판단할 근거가 코드 자체에는 없으며, 매번 별도로 수동 측정해야 한다.

확장성 문제. RTMPose 외 다른 모델 (HaMeR, WiLoR) 과 비교하려면 추론 호출 부분을 포함한 스크립트 전체를 복사해야 한다. 실제로 이런 방식으로 만들어진 것으로 보이는 거의 동일한 스크립트가 코드베이스에 5 개 이상 존재한다 (`compare_new.py`, `compare_new2.py`, `compare_26.py`, `full_compare.py`, `compare_angles.py` 등). 이는 “실험 조건 변경이 어려운 부분”에 정확히 해당하는 문제이며, 코드 중복으로 인한 유지보수 비용 증가로 이어진다.

재현성 문제. 경로, 디바이스, GT 컬럼 목록이 모두 하드코딩되어 있어, 다른 환경이나 다른 녹화 데이터로 평가하려면 스크립트 본문을 직접 수정해야 한다. 어떤 값을 어디서 바꿔야 하는지 코드를 끝까지 읽어야 알 수 있다.

7.3 적용한 수업 개념

본 과제에서는 다음 세 가지 기법을 적용하였다.

(C) Class 기반 구조 개선 및 단일책임원칙. 설정값을 `EvalConfig`(`@dataclass(frozen=True, slots=True)`) 로, 모델 추론 인터페이스를 `PoseEstimator`(`typing.Protocol`) 로, 평가 로직을 `Evaluator` 클래스로 분리하였다. `frozen=True` 는 평가 도중 설정값이 실수로 변경되는 것을 막고, `slots=True` 는 불필요한 `__dict__` 오버헤드를 줄인다. `Protocol` 을 사용한 이유는 `RTMPose`, `HaMeR`, `WiLoR` 가 입력 (이미지) 과 출력 (21 개 키포인트) 의 형태는 같지만 내부 구현이 완전히 다르기 때문에, 상속 기반 추상클래스보다 구조적 타이핑 (structural typing) 이 더 적합하다고 판단했기 때문이다. 이 분리를 통해 `Evaluator(config, estimator)` 에서 `estimator` 만 교체하면 다른 모델로 평가할 수 있게 되어, 5.1 에서 지적한 “스크립트 전체 복붙” 문제를 직접 해결한다.

(D) Decorator 기반 개선. `@timed("inference")` 로 추론 호출마다 걸린 시간을 `self.timings` 에 자동 기록하고, `@cached_by_arg("_kp_cache")` 로 동일한 이미지 경로에 대한 중복 추론을 방지하였다. 두 데코레이터를 `@cached_by_arg` 바깥, `@timed` 안쪽 순서로 스택했는데, 이는 캐시 적중 시 `timed` 의 본문 (원본 함수 호출) 이 실행되지 않아 타이밍 기록 자체가 “실제 GPU 추론이 일어난 경우만” 포함하도록 설계한 것이다. 즉 캐싱의 효과를 별도 측정 없이 타이밍 기록 자체로 증명할 수 있다. 두 데코레이터 모두 `functools.wraps` 를 사용하여 원본 메서드의 이름과 시그니처를 보존하였다.

(A) 자료구조 및 반복계산 개선. 12 개 관절의 (p1, p2, p3) 인덱스 triple 을 `FINGER_JOINT_TRIPLES: dict[str, list[tuple[int, int, int]]]` 로 명시적으로 정의하여, 원본에서 호출 순서로만 암묵적으로 표현되던 “관절-손가락” 대응 관계를 코드 구조 자체로 드러냈다. 또한 `calc_angle` 을 관절마다 12 번 호출 하던 것을, 손가락당 3 개 관절을 `np.einsum` 으로 한 번에 계산하는 벡터화된 `_batch_angles` 함수로 대체 하였다. 이는 “같은 계산을 반복하는 부분”을 줄이는 동시에, 결과를 `dict[str, np.ndarray]` 로 그룹화 해 이후 디버깅 (손가락별 오차 확인) 도 더 쉽게 만든다.

7.4 개선 과정

수정 전 구조: 단일 스크립트, 전역 변수, 12 회 개별 함수 호출, 계측 없음.

수정 후 구조: `config.py`(설정), `pose_estimator.py`(모델 인터페이스), `decorators.py`(timing/caching), `angle_calc.py`(각도 계산), `evaluator.py`(평가 로직 + 결과 리포트), `main.py`(엔트리포인트) 로 분리된 6 개 모듈.

주요 변경 사항: 1. 경로/컬럼/디바이스를 `EvalConfig` `dataclass` 로 이동 2. `init_model/inference_topdown` 호출을 `RTMPoseEstimator.predict()` 로 캡슐화 3. `calc_angle` 12 회 개별 호출 → `_batch_angles` 벡터화 + `FINGER_JOINT_TRIPLES` dict 4. 추론 메서드에 `@timed`, `@cached_by_arg` 적용 5. MAE 계산과 리포트 출력을 `EvalResult` 클래스로 분리 (pandas DataFrame 기반)

변경 이유: 가장 큰 동인은 “모델 교체 시 전체 복붙”이라는 반복된 패턴이었다. `PoseEstimator Protocol` 을 먼저 설계하고, 그 인터페이스를 만족하도록 나머지 구조를 역으로 설계하였다.

고려했지만 적용하지 않은 대안: `PoseEstimator` 를 `abc.ABC` 상속으로 구현하는 방안도 고려했으나, `RTMPose`/`HaMeR`/`WiLoR` 는 서로 다른 라이브러리 (`mmpose`, 자체 `HaMeR` 코드베이스) 에 속해 있어 공통 베이스 클래스를 상속시키기보다 `Protocol` 로 덕 타이핑하는 것이 더 느슨한 결합 (loose coupling) 을 만든다고 판단하여 `Protocol` 을 선택하였다. 또한 generator 기반으로 프레임을 스트리밍 처리하는 방안도 검토하였으나, 현재 평가 프레임 수 (123 장) 가 메모리 문제를 일으킬 규모가 아니라고 판단해 이번 과제 범위에서는 제외하였다 (7.6 에서 향후 개선 방향으로 언급).

Trade-off: `Protocol` 기반 구조는 모델 1 개만 쓰는 현재 시점에서는 코드량이 늘어나는 오버헤드가 있다. 그러나 동일 연구에서 `HaMeR`/`WiLoR` 와의 비교 평가가 곧 필요하다는 점을 고려하면, 이 오버헤드는 향후

작업량 감소로 상쇄될 것으로 판단한다.

7.5 최적화 결과

정확도 일치 검증. 리팩토링 전후 동일한 데이터 (123 프레임, CSV 기준 4 프레임만 GT 매칭) 에 대해 MAE 를 비교한 결과, 원본과 리팩토링 버전 모두 전체 평균 MAE **34.82°** 로 동일하게 측정되어, 구조 변경이 계산 결과에 영향을 주지 않았음을 확인하였다.

CPU vs GPU 추론 시간.

환경	평균 추론 시간/frame	표준편차	fps
CPU	2833.06 ms	± 929.96 ms	0.4
GPU (RTX 4070)	86.x ms (warm-up 후, 아래 표 참조)	$\pm 0.3\sim 0.6$ ms	약 11.6

GPU 는 CPU 대비 약 21~33 배 빠른 추론 시간을 보였다.

입력 크기별 추론 시간 (GPU, warm-up 1 회 후 측정, 각 3 회 반복).

입력 프레임 수	총 시간 평균	총 시간 표준편차	frame 당 시간
30	2.605 s	± 0.018 s	86.8 ms $\pm$ 0.6 ms
61	5.276 s	± 0.022 s	86.5 ms $\pm$ 0.4 ms
123	10.558 s	± 0.032 s	85.8 ms $\pm$ 0.3 ms

프레임 수가 약 2 배씩 증가할 때 총 시간도 거의 정확히 2 배씩 증가하고 (2.605 $\rightarrow$ 5.276 $\rightarrow$ 10.558), frame 당 평균 시간은 85.8~86.8ms 로 거의 일정하다. 이는 RTMPose 추론이 입력 크기에 대해 $O(n)$ 으로 선형 스케일링하며, 프레임 간 간섭이나 추가 오버헤드가 거의 없음을 보여준다. 표준편차가 모두 1ms 이하로 매우 작다는 점도 측정의 안정성을 뒷받침한다.

캐싱 효과. @cached_by_arg 를 적용한 이후, 동일 이미지 경로에 대한 재호출은 GPU 추론을 거치지 않고 캐시된 키포인트를 즉시 반환한다. 같은 프레임 집합으로 Evaluator.run() 을 캐시를 비우지 않고 두 번 호출할 경우, 두 번째 호출의 self.timings["inference"] 에는 새로운 기록이 추가되지 않는다 (캐시 적용이므로 @timed 의 본문이 실행되지 않음). 즉 동일 입력에 대한 반복 평가 (예: MAE 계산 로직만 수정한 뒤 재실행) 에서 추론 비용이 정확히 0 으로 줄어든다.

구조 개선. 원본은 1 개 파일·2 개 함수로 구성되어 있었으나, 리팩토링 후 6 개 모듈로 역할이 분리되었다. 모델 교체에 필요한 수정 범위는 원본 기준 “스크립트 전체”에서 리팩토링 후 “pose_estimator.py 에 클래스 1 개 추가”로 축소되었다.

7.6 결과 해석 및 한계

기대했던 두 가지 개선—(1) 모델 교체 시 수정 범위 축소, (2) 캐싱을 통한 반복 평가 비용 절감—은 모두 실제로 확인되었다. 특히 입력 크기별 벤치마크에서 frame 당 시간이 거의 완벽하게 일정하게 유지된 것은, 벡터화 (_batch_angles) 와 구조 분리가 추가 오버헤드를 거의 만들지 않았다는 근거로 해석할 수 있다. 즉 “코드를 여러 클래스/모듈로 나누면 오버헤드가 생긴다”는 일반적 우려가, 이 경우에는 GPU 추론 자체의 비용 (약 86ms) 이 Python 레벨의 함수 호출/객체 생성 오버헤드 (수 μ s~ 수십 μ s 단위) 를 압도하기 때문에 무시할 수준이었다.

다만 한계도 명확하다. 첫째, 본 연구의 평가 데이터셋이 123 프레임으로 작아, 입력 크기를 훨씬 크게 (예: 수천 프레임) 늘렸을 때도 동일한 선형성이 유지되는지는 확인하지 못했다. 둘째, 측정된 fps(약 11.6)는 과거에 별도로 측정해 둔 모델 자체의 처리량 (약 88fps) 보다 낮는데, 이는 본 벤치마크가 이미지 디코딩 (cv2.imread) 과 결과 후처리까지 포함한 end-to-end 시간을 측정한 반면, 88fps 는 모델 forward pass 만을 측정한 수치였기 때문으로 추정된다. 셋째, 이번 레코딩 (CSV) 에서는 123 프레임 중 4 프레임만 Manus GT 와 매칭되어 MAE 평가 자체의 표본 수가 매우 작다. 이는 코드의 문제가 아니라 프레임 폴더와 CSV 레코딩 간의 시간 정렬 (매핑 배수, 녹화 시점 차이 등) 문제로 보이며, 추후 데이터 정합성을 다시 점검할 필요가 있다. 넷째, generator 기반 스트리밍 처리는 이번 과제 범위에서는 다루지 않았는데, FreiHAND(약 32,500 프레임) 처럼 훨씬 큰 데이터셋에 동일한 평가 파이프라인을 적용할 경우, 모든 키포인트를 리스트에 누적하는 현재 방식이 메모리 문제를 일으킬 수 있어 향후 개선이 필요하다.

종합하면, 이번 리팩토링은 정확도 (MAE) 를 그대로 유지하면서 구조적 재사용성과 계측 가능성을 크게 개선했으며, 성능 측정 결과는 코드 변경이 실행 시간에 부정적 영향을 주지 않았음을 정량적으로 보여준다.